

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA1403

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS: 5

**Duration :60 Mins
On Class
Total Marks:50**

Annexure A

ANALYTICAL PROBLEM SOLVING

Code of Conduct:

I Pooja Sree . M (192521403) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Pooja Sree. M

Signature of the student

C01-ASSESSMENT TOOL 1

ANALYTICAL PROBLEM SOLVING

1.) Demonstrate how a source program is processed through phases of compiler

Statement:
 $W = X * Y - Z + 20$

Lexical Analysis

converts source code into tokens and remove unnecessary spaces

$\langle id_1 \rangle \langle = \rangle \langle id_2 \rangle \langle * \rangle \langle id_3 \rangle \langle - \rangle \langle id_4 \rangle \langle + \rangle \langle 20 \rangle$

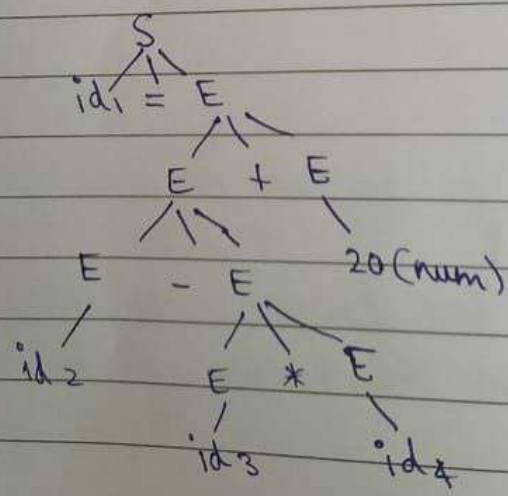


Syntax analysis

checks whether the token sequence follows grammar rules of language

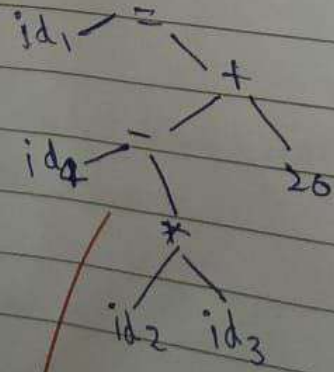
$S \rightarrow id = E$

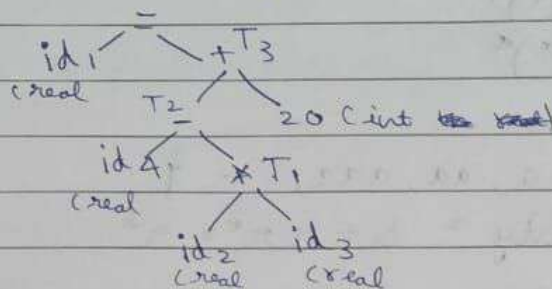
$E \rightarrow E * E / E - E / E + E / id / num$



Parse Tree

Syntax Tree



Semantic AnalyzerIntermediate code generator

$$T_1 = id_2 * id_3$$

$$T_2 = T_1 - id_4$$

$$T_3 = T_2 + 20$$

$$id_1 = T_3$$

↓

Code optimizer

$$T_1 = id_2 * id_3$$

$$T_2 = T_1 - id_4$$

$$id_1 = T_2 + 20$$

↓

Code generator

MOV R1, id2

MOV R2, id3

MUL R1, R2

MOV R3, id4

SUB R1, R3

ADD R1, #20

MOV id1, R1

2) CONSTRUCTION OF REGULAR EXPRESSION:

i) $1(0+1)^*1+1$ ii) $(0+1)^*01(0+1)^*$ iii) 1^*01^* iv) $(0+1)^*1(0+1)^*1(0+1)^*$ v) $\epsilon + (0+1)^*1$

3.) Identifying string Pattern

i.) $(\epsilon + aa^*)^*$

$(\epsilon + a^*)^*$

$= a^*$

$= \{ \epsilon, a, aa, aaa, \dots \}$

Identity $= \epsilon + aa^* = a^*$

ii.) $(a^* + b^*)^* abb$

$(a + b)^* abb$

$= \{ abb, aabb, baab, aababb, baabab, \dots \}$

iii.) $(a^* b^*)^* aba$

$(a + b)^* aba$

$= \{ aba, aaba, baba, aaababa, bbaaba, \dots \}$

iv.) $(a^* + ab^*)^* (aba)^* (a + ab)^*$

$= \{ \epsilon, a, ab, aa, aab, aba, abab, \dots \}$

v.) $(aba)^* + (a^* b^*)^*$

$= \{ \epsilon, aba, abaaba, a, b, aabb, \dots \}$

4.)

i.)

```
%s
#include <stdio.h>
```

```
%c
```

```
%f
```

```
if
```

```
else
```

```
printf { printf ("%s is a keyword", ytext); }
[A-Za-z][A-Za-z0-9_]* { printf ("%s is an identifier", ytext); }
```

```
[0-9]+ { printf ("%s is a constant", ytext); }
```

```
[+|-|*|/] { printf ("%s is an operator", ytext); }
```

```
[(){}]; { printf ("%s is a delimiter", ytext); }
```

```
%s
```

```

int main()
{
    printf("\n Enter a String:");
    yylex();
}

int yylex()
{
    return 1;
}

```

i) Thus, lexical analyzer correctly recognizes keywords separately & identifies all other valid name as identifiers

5.)

i) $(0^*1^*)^* = (0+1)^*$

$$(0^*1^*)^* = \{\epsilon, 0, 1, 00, 01, 10, 11, \dots\}$$

$$(0+1)^* = \{\epsilon, 0, 1, 01, 00, 10, 11, \dots\}$$

$$\text{LHS} = \text{RHS}$$

ii) $(0+1)^* (0+1)^* = (0+1)^*$

$$(0+1)^* (0+1)^* = \{\epsilon, 0, 1, 11, 00, \dots\}$$

$$(0+1)^* = \{\epsilon, 0, 1, 11, 00, \dots\}$$

$$\text{LHS} = \text{RHS}$$

iii) $(0^*)^* = 0^*$

$$(x^*)^* = x^*$$

$$\text{LHS} = \text{RHS}$$

iv) $(0+1)^* 0 (0+1)^* + (0+1)^* 1 (0+1)^* = (0+1)^*$

$$(0+1)^* (0 (0+1)^* + 1 (0+1)^*) = \epsilon$$

$$= (0+1)^* (0+1)^*$$

$$\text{LHS} = \text{RHS}$$

v) $\epsilon + 0(0)^* = 0^*$

$$\epsilon + x(x^*) = x^*$$

, By identity $\text{LHS} = \text{RHS}$

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation